

LINUX LAB ASSIGNMENT

Aditi Biju
2611100690027

Part A – Basic grep and Filtering

Display all lines containing the word failed.

```
msis@main:~/lab4$ grep 'Failed' log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:29:58] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:17] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Display all lines containing failed, ignoring case.

```
msis@main:~/lab4$ grep -i 'failed' log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:29:58] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:17] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Display all lines that contain either Failed or Denied.

```
msis@main:~/lab4$ grep -E 'Failed|Denied' log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:29:58] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:17] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Display all lines that do not contain the word INFO.

```
msis@main:~/lab4$ grep -v 'INFO' log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:13:02] WARNING Multiple authentication failures from 203.0.113.45
[2025-08-18 08:15:44] ERROR Connection denied from 198.51.100.77 port 9001
[2025-08-18 08:16:05] Suspicious connection from 198.51.100.77 to evilc2.ddns.net:9001
[2025-08-18 08:16:19] Suspicious outbound request https://evilc2.ddns.net/update.php
```

Display all lines containing the exact word root.

```
msis@main:~/lab4$ grep -w 'root' log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:27:24] Suspicious crontab created for user root
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
```

Display all occurrences of the word attack along with their line numbers.

```
msis@main:~/lab4$ grep -ni 'attack' log.txt
17:[2025-08-18 08:20:10] WARNING Email received from attacker123@phishing-example.com
```

Count the number of lines containing ERROR.

```
msis@main:~/lab4$ grep -c 'ERROR' log.txt
10
```

Display only the filenames of files whose contents contain the word malware, assuming multiple log files are available.

```
msis@main:~/lab4$ grep -l 'malware' *.txt
log.txt
```

Display the first five lines containing Suspicious.

```
msis@main:~/lab4$ grep 'Suspicious' log.txt | head -n 5
[2025-08-18 08:16:05] Suspicious connection from 198.51.100.77 to evilc2.ddns.net:9001
[2025-08-18 08:16:19] Suspicious outbound request https://evilc2.ddns.net/update.php
[2025-08-18 08:18:20] Suspicious file download evil.exe from http://malware-test.example.org/evil.exe
[2025-08-18 08:19:45] Suspicious file invoice_2025.pdf downloaded from https://phishing-example.com/invoice_2025.pdf
[2025-08-18 08:20:51] Suspicious email phishing.alert@evilmail.ru contained malicious attachment
```

Display the last five failed-login entries.

```
msis@main:~/lab4$ grep -Ei 'failed.*login|login.*failed' log.txt| tail -5
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Part B – Regular Expression Extraction

Extract all IPv4 addresses from the log.

```
msis@main:~/lab4$ grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' log.txt
192.168.10.5
192.168.10.21
203.0.113.45
203.0.113.45
203.0.113.45
203.0.113.45
```

Extract all IP addresses belonging to the 10.x.x.x private network.

```
msis@main:~/lab4$ grep -Eo '\b10(\. [0-9]{1,3}){3}\b' log.txt
10.10.5.23
10.20.30.40
10.1.2.3
10.1.2.3
10.1.2.3
10.1.2.3
```

Extract all addresses belonging to the 192.168.x.x network.

```
msis@main:~/lab4$ grep -Eo '\b192\.168(\. [0-9]{1,3}){2}\b' log.txt
192.168.10.5
192.168.10.21
192.168.10.21
192.168.1.50
192.168.10.5
192.168.10.5
192.168.10.21
192.168.10.21
```

Extract all email addresses.

```
msis@main:~/lab4$ grep -Eo '[[[:alnum:]]._%+-]+@[[:alnum:]].-]+\.[[:alpha:]]{2,}' log.txt
attacker123@phishing-example.com
security.team@example.com
phishing.alert@evilmail.ru
student2025@university.in
```

Extract only email addresses whose domain ends in .com.

```
msis@main:~/lab4$ grep -Eo '[[[:alnum:]]._%+-]+@[[:alnum:]].-]+\.[[:alpha:]]{2,}' log.txt
attacker123@phishing-example.com
security.team@example.com
```

Extract email addresses whose username contains at least one digit.

```
msis@main:~/lab4$ grep -Eo '[[[:alnum:]]._%+-]*[0-9][[:alnum:]]._%+-]+@[[:alnum:]].-]+\.[[:alpha:]]{2,}' log.txt
attacker123@phishing-example.com
student2025@university.in
```

Extract all URLs beginning with http:// or https://.

```
msis@main:~/lab4$ grep -Eo 'https?://[^[[:space:]]]+' log.txt
https://evilc2.ddns.net/update.php
http://malware-test.example.org/download.php
http://malware-test.example.org/evil.exe
https://phishing-example.com/invoice_2025.pdf
https://target.example.com/login.php
https://www.example.com/index.html
https://shop.example.com/admin.php
```

Extract only HTTPS URLs.

```
msis@main:~/lab4$ grep -Eo 'https?://[^[[:space:]]+' log.txt
https://evilc2.ddns.net/update.php
https://phishing-example.com/invoice_2025.pdf
https://target.example.com/login.php
https://www.example.com/index.html
https://shop.example.com/admin.php
https://portal.university.in/login.php
https://updates.bad-domain.ru/update.php
https://updates.bad-domain.ru/update.php
https://files.example.com/upload.php
https://command.bad-domain.in/backdoor.exe
```

Extract URLs ending with .php.

```
msis@main:~/lab4$ grep -Eo 'https?://[^\s:]]+\.\php\b' log.txt
https://evilc2.ddns.net/update.php
http://malware-test.example.org/download.php
https://target.example.com/login.php
https://shop.example.com/admin.php
http://legacy.example.net/login.php
https://portal.university.in/login.php
https://updates.bad-domain.ru/update.php
https://updates.bad-domain.ru/update.php
https://files.example.com/upload.php
```

Extract all CVE identifiers in the form:CVE-YYYY-NNNN

```
msis@main:~/lab4$ grep -Eo 'CVE-[0-9]{4}-[0-9]{4}' log.txt
CVE-2025-1234
CVE-2024-3094
CVE-2025-9876
CVE-2023-4448
CVE-2025-5432
CVE-2025-5432
```

Extract only CVEs reported for the year 2025.

```
msis@main:~/lab4$ grep -Eo 'CVE-2025-[0-9]{4}' log.txt
CVE-2025-1234
CVE-2025-9876
CVE-2025-5432
CVE-2025-5432
```

Extract all 32-character hexadecimal MD5 hashes.

```
msis@main:~/lab4$ grep -Eio '\b[0-9a-f]{32}\b' log.txt
44d88612fea8a8f36de82e1278abb02f
e2fc714c4727ee9395f324cd2e7f331f
5d41402abc4b2a76b9719d911017c592
```

Extract all 40-character hexadecimal SHA-1 hashes.

```
msis@main:~/lab4$ grep -Eio '\b[0-9a-f]{40}\b' log.txt
3395856ce81f2b7382dee72602f798b642f14140
```

Extract all 64-character hexadecimal SHA-256 hashes.

```
msis@main:~/lab4$ grep -Eio '\b[0-9a-f]{64}\b' log.txt
275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f
3a7bd3e2360a3d80a9d0b7b93d7f90d7c0b8c9a3e9f1d2c4b5a6978877665544
```

**Extract all timestamps having the format:
YYYY-MM-DD HH:MM:SS**

```
msis@main:~/lab4$ grep -Eo '\b[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}\b' log.txt
2025-08-18 08:10:01
2025-08-18 08:11:14
2025-08-18 08:12:03
2025-08-18 08:12:10
2025-08-18 08:12:17
2025-08-18 08:13:02
2025-08-18 08:14:33
2025-08-18 08:15:44
2025-08-18 08:16:05
2025-08-18 08:16:19
2025-08-18 08:17:31
2025-08-18 08:18:07
2025-08-18 08:18:20
2025-08-18 08:18:32
```

Part C – Anchors and Regex Patterns

Display all lines beginning with a timestamp.

```
grep -E '^'[0-9]{4}-[0-9]{2}-[0-9]{2}' security.log
```

Display all lines beginning with the word Suspicious.

```
grep '^Suspicious' security.log
```

Display lines ending with a numeric value.

```
grep -E '[0-9]+$' security.log
```

Display lines beginning with ERROR or WARNING.

```
grep -E '^(ERROR|WARNING)' security.log
```

Display lines whose first character is a digit.

```
grep -E '^[0-9]' security.log
```

Find lines ending in .exe.

```
grep -E '\.exe$' security.log
```

Find lines containing filenames ending in .exe, .sh, or .pdf.

```
grep -E '\.(exe|sh|pdf)' security.log
```

Identify lines containing the exact word root, but not words such as rooted.

```
grep -E '\broot\b' security.log
```

Display blank lines in the file.

```
grep -E '^$' security.log
```

Display all nonblank lines.

```
grep -E '.*' security.log
```

Part D – Linux Filters and Pipelines

Extract all IPv4 addresses, sort them, and remove duplicates.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | sort -u
```

Count the number of unique IPv4 addresses present in the log.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | sort -u | wc -l
```

Extract all IP addresses and display them in sorted order.

```
Grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | sort
```

Determine which IP addresses appear more than once.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | sort
```

Determine the IP address that occurs most frequently.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | sort | uniq -c | sort -nr | head -1
```

Extract all usernames from authentication entries and display unique usernames.

```
grep -Ei 'authentication|login' security.log | grep -Eo 'username[=:]^[^ ]+' | cut -d '=' -f2 | sort -u
```

Count the number of authentication attempts associated with each username.

```
grep -Ei 'authentication|login' security.log | grep -Eo 'username[=:]^[^ ]+' | cut -d '=' -f2 | sort | uniq -c  
| sort -nr
```

Extract all HTTP status codes and determine how many times each status code occurs.

```
grep -Eo '\b[1-5][0-9]{2}\b' security.log | sort | uniq -c | sort -nr
```

Display the five most frequently occurring IP addresses.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | sort | uniq -c | sort -nr | head -5
```

Display the three most frequently occurring usernames.

```
grep -Ei 'authentication|login' security.log | grep -Eo 'username[=:]^[^ ]+' | cut -d '=' -f2 | sort | uniq -c  
| sort -nr | head -3
```

Extract all destination ports and display them in numerical order.

```
grep -Eo 'port[=:]?[0-9]+' security.log | grep -Eo '[0-9]+' | sort -n
```

Remove duplicate port numbers.

```
grep -Eo 'port[=:]?[0-9]+' security.log | grep -Eo '[0-9]+' | sort -n -u
```

Count the number of unique destination ports.

```
grep -Eo 'port[=:]?[0-9]+' security.log | grep -Eo '[0-9]+' | sort -n -u | wc -l
```

Extract all file extensions appearing in the log and determine their frequency.

```
grep -Eo '\.[A-Za-z0-9]+([[:space:]]|$)' security.log | tr -d ' ' | sort | uniq -c | sort -nr
```

Find the five most frequently occurring domains.

```
grep -Eo '\b([A-Za-z0-9-]+\.)+[A-Za-z]{2,}\b' security.log | sort | uniq -c | sort -nr | head -5
```

Part E – Combining grep Conditions

Display lines containing both an IP address and the word Failed.

```
grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}' security.log | grep 'Failed'
```

Display lines containing both root and Failed.

```
grep 'root' security.log | grep 'Failed'
```

Display lines containing either download or upload.

```
grep -Ei 'download|upload' security.log
```

Display suspicious entries containing either .exe or .sh.

```
grep -i 'Suspicious' security.log | grep -Ei '\.exe|\.sh'
```

Display lines containing a CVE identifier and an IP address.

```
grep -E 'CVE-[0-9]{4}-[0-9]{4,}' security.log | grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}'
```

Display lines containing both a filename and a hash value.

```
grep -Ei '\.[A-Za-z0-9]+.*[0-9a-fA-F]{32,}' security.log
```

Display entries containing an IP address followed by a port.

```
grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}:[0-9]+' security.log
```

Display HTTP requests containing both a URL and status code 404.

```
grep -E 'https?:/' security.log | grep -E '\b404\b'
```

Display authentication failures but exclude entries generated from 127.0.0.1.

```
grep -Ei 'authentication.*fail|failed.*login' security.log | grep -v '127\.\0\.\0\1'
```

Display attack-related entries but exclude informational (INFO) messages.

```
grep -Ei 'attack' security.log | grep -v 'INFO'
```

Part F – Negative Matching

Extract all IPv4 addresses except those beginning with 127.

```
grep -Eo '\b(?!127\.)([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log
```

Display all log entries except successful login events.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | grep -v '^127\.'
```


Display all entries that do not contain INFO, DEBUG, or SUCCESS.

```
grep -Eiv 'successful login|login successful' security.log
```

Find HTTP requests whose status code is not 200.

```
grep -E 'https?:/' security.log | grep -Ev '\b200\b'
```

Display suspicious traffic excluding connections to 192.168.x.x.

```
grep -i 'Suspicious' security.log | grep -Ev '192\.168\.'
```

Extract email addresses except those belonging to the organization's domain.

```
grep -Eo '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' security.log | grep -v '@example\.com$'
```

Part G – Redirection

Extract all IPv4 addresses and save them into: ips.txt

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log > ips.txt
```

Extract all email addresses and save them into: emails.txt

```
grep -Eo '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' security.log > emails.txt
```

Extract all URLs and save them into: urls.txt

```
grep -Eo 'https?:/[^\[:space:]]+' security.log > urls.txt
```

Extract all hashes and save them into: hashes.txt

```
grep -Eo '\b[0-9a-fA-F]{32}\b\b[0-9a-fA-F]{40}\b\b[0-9a-fA-F]{64}\b' security.log > hashes.txt
```

Extract failed-login entries and save them into: failed_login.txt

```
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security.log > failed_login.txt
```

Extract suspicious entries and append them to: incident_report.txt

```
grep -i 'Suspicious' security.log >> incident_report.txt
```

Store unique IP addresses in: unique_ips.txt

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | sort -u > unique_ips.txt
```

Store IP-frequency statistics in: ip_frequency.txt

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security.log | sort | uniq -c | sort -nr > ip_frequency.txt
```

Redirect normal output to results.txt and errors to errors.txt.

```
grep -E 'ERROR|WARNING' security.log > results.txt 2> errors.txt
```

Extract all domains from the log entries containing Domain accessed, convert the domain names to lowercase, sort them alphabetically, remove duplicates, and save the result in domains.txt.

```
grep 'Domain accessed' security.log | grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}' | tr '[:upper:]' '[:lower:]' | sort -u > domains.txt
```

identify all IP addresses involved in failed-login attempts. Convert the output into a clean list containing one unique IP per line and save it as failed_login_ips.txt.

```
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security.log | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort -u > failed_login_ips.txt
```

Combining filters

Extract all failed-login usernames and save them into: failed_users.txt

```
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security.log | grep -Eo 'username[=:]^[^ ]+' | cut -d'=' -f2 | sort -u > failed_users.txt
```

Extract all domains and save them in lowercase into: domains.txt

```
grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}' security.log | tr '[:upper:]' '[:lower:]' > domains.txt
```

Extract all MD5 hashes, convert them to uppercase, and save them into: md5_hashes.txt

```
grep -Eo '\b([0-9a-fA-F]{32})\b' security.log | tr '[:lower:]' '[:upper:]' > md5_hashes.txt
```

Extract source IP addresses from failed-login records, sort them, remove duplicates, and redirect the result to: failed_ips.txt

```
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security.log | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort -u > failed_ips.txt
```

Create a file named clean_log.txt in which all square brackets are removed from the original log.

```
sed 's/[//g; s/]/g' security.log > clean_log.txt
```

Create normalized_log.txt by converting the complete log file to lowercase.

```
tr '[:upper:]' '[:lower:]' < security.log > normalized_log.txt
```

Extract all domain names, convert them to lowercase, sort them, remove duplicates, and save them to unique_domains.txt.

```
grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}' security.log | tr '[:upper:]' '[:lower:]' | sort -u > unique_domains.txt
```

Find all failed-login entries and extract only the usernames.

```
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security.log | grep -Eo 'username[=:]^[^ ]+' | cut -d'=' -f2
```

Find all failed-login entries, extract only the source IP addresses, and display them in sorted order.

```
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security.log | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort
```

Extract IP addresses associated with failed logins and determine the unique IPs.

```
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security.log | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort -u
```

Extract usernames involved in failed logins, convert them to uppercase, and remove duplicate values.

```
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security.log | grep -Eo 'username[=:][^ ]+' | cut -d '=' -f2 | tr '[:lower:]' '[:upper:]' | sort -u
```

Find all lines containing Domain accessed, extract only the domain names, and convert them to lowercase.

```
grep 'Domain accessed' security.log | grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}' | tr '[:upper:]' '[:lower:]'
```

Find all lines containing MD5=, extract only the MD5 hash values, and convert hexadecimal letters to uppercase.

```
grep 'MD5=' security.log | grep -Eo 'MD5=[0-9a-fA-F]{32}' | cut -d '=' -f2 | tr '[:lower:]' '[:upper:]'
```

Find entries containing User-Agent:, extract the user-agent field, and convert it to lowercase.

```
grep 'User-Agent:' security.log | sed 's/.*User-Agent:[[:space:]]*//' | tr '[:upper:]' '[:lower:]'
```

Find all CVE-related lines, extract the CVE identifiers, and convert them to lowercase.

```
grep -Eo 'CVE-[0-9]{4}-[0-9]{4,}' security.log | tr '[:upper:]' '[:lower:]'
```

Find all entries containing .exe, extract the relevant filename, and convert it to uppercase.

```
grep -Eo '[A-Za-z0-9_./-]+\.[exe]' security.log | tr '[:lower:]' '[:upper:]'
```

Extract all domains from Domain accessed entries, convert them to uppercase, sort them, and remove duplicates.

```
grep 'Domain accessed' security.log | grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}' | tr '[:lower:]' '[:upper:]' | sort -u
```